MP2 Part 1: Build the API Foundation

In Part 1, you will build a FastAPI application that accepts a CSV file, stores its data in memory, and returns stored rows through HTTP requests.

Suggested pacing: After Class 12, focus on setup, logging, and response models. After Class 13, complete storage and the endpoints, then test the API.

Project Structure

At the end of Part 1, your repository should contain:

```
mp2-rest-api/
├── .gitignore
├── requirements.txt
├── main.py
├── src/
│   ├── __init__.py
│   ├── models.py
│   ├── storage.py
│   └── utils.py
└── fixtures/
    └── sample_data.csv
```

Commit the sample data.

Keep `venv/`, `__pycache__/`, `.pytest_cache/`, and generated files *out of Git*.

Getting Started

1. Create a local `mp2-rest-api` project and initialize Git. Then create an empty repository on GitHub and connect your local repository to it.
2. Create and activate a virtual environment, then install:

```
python -m pip install fastapi "uvicorn[standard]" python-multipart pandas
pytest httpx
python -m pip freeze > requirements.txt
```

Commit your progress as you complete each section.

1. Create `src/`

1. Create `src/utils.py`

Copy `setup_logging()` from your MP1 `src/utils.py`. Keep its existing behavior and logging format.

2. Create `src/models.py`

Complete these response models:

```
from pydantic import BaseModel


class HealthResponse(BaseModel):
    status: str
    version: str


class DatasetInfo(BaseModel):
    # TODO: id (int), filename (str), rows (int),
    # columns (int), column_names (list[str])
    pass
```

Use these models as the corresponding endpoint's `response_model`.

3. Create `src/storage.py`

Complete the following class:

```
import logging

logger = logging.getLogger(__name__)


class DatasetStorage:
    def __init__(self):
        self._store = {}
        self._counter = 0

    def save(self, df, filename):
        """Store a copy of the data and return a new integer ID."""
        pass

    def get(self, dataset_id):
        """Return a copy of the stored DataFrame, or None."""
        pass

    def get_metadata(self, dataset_id):
        """Return the dataset metadata, or None."""
        pass

    def update(self, dataset_id, df):
        """Replace an existing dataset; return True, or False if
missing."""
        pass
```

```
    def clear(self):
        """Clear all data and reset the ID counter."""
        self._store.clear()
        self._counter = 0
```

Requirements:

- Assign IDs starting at `1` and increment for each upload.
- Store the filename and a copy of the DataFrame for each ID.
- Return metadata matching the fields in `DatasetInfo`.
- Calculate metadata from the current DataFrame so it remains accurate after processing.
- Return copies from `get()` so changes do not affect storage until `update()` is called.
- Log storage operations at `DEBUG`.

4. Add `src/__init__.py`

2. Create `main.py`

First, update `src/__init__.py` to expose the selected models, class, and function:

```
from .models import DatasetInfo, HealthResponse
from .storage import DatasetStorage
from .utils import setup_logging
```

Create `main.py` in the repository root using this starter code:

```
import io
import json
import logging
from pathlib import Path
import pandas as pd
from fastapi import FastAPI, File, HTTPException, Query, UploadFile
from src import DatasetInfo, DatasetStorage, HealthResponse, setup_logging

setup_logging(verbose=True)
logger = logging.getLogger(__name__)

app = FastAPI(title="Data Processing API", version="1.0.0")
storage = DatasetStorage()

@app.get("/health", response_model=HealthResponse)
def health_check():
    """Return the application status and version."""
    pass

@app.post("/upload", response_model=DatasetInfo)
async def upload_dataset(file: UploadFile = File(...)):
    """Read and store a CSV upload."""
```

```
    pass

@app.get("/datasets/{dataset_id}")
def get_dataset(dataset_id: int, limit: int = Query(default=100, ge=1)):
    """Return up to limit rows from a stored dataset."""
    pass
```

1. `GET /health`

Return:

```
{"status": "ok", "version": "1.0.0"}
```

2. `POST /upload`

- Check the filename extension with `Path(file.filename or "").suffix.lower()`.
- Accept CSV only. Raise `HTTPException` with status `400` for another extension.
- Read the contents using `await file.read()`.
- Parse the bytes using `pd.read_csv(io.BytesIO(contents))`.
- Catch `pd.errors.EmptyDataError`, `pd.errors.ParserError`, and `UnicodeDecodeError`. Log a `WARNING` and raise `HTTPException` with status code `400` and a helpful message.
- Save the DataFrame and return its metadata as `DatasetInfo`.
- Log the assigned ID, filename, and row count at `INFO`.

3. `GET /datasets/{dataset_id}`

- Retrieve the dataset. If missing, log a `WARNING` and raise status `404`.
- Return up to `limit` rows, preserving the stored row order.
- Include `id`, `data`, and `total_rows` in the response. `total_rows` is the number of rows in the stored dataset, not the number returned.
- Leave the stored dataset unchanged.
- Log the ID and returned row count at `INFO`.

To produce JSON records with missing values represented as `null`, use:

```
records = json.loads(df.head(limit).to_json(orient="records"))
```

For `limit=2`, a sample response is:

```
{
  "id": 1,
  "data": [
    {"student_id": 1, "score": "80"},
    {"student_id": 2, "score": "90"}
  ],
```

```
  "total_rows": 6
}
```

3. Create the Sample Data

Create `fixtures/`.

Save this as `fixtures/sample_data.csv`:

```
student_id,score
1,80
2,90
2,90
3,
4,unknown
5,85
```

4. Run the API

From the repository root:

```
python -m uvicorn main:app --reload
```

Open http://127.0.0.1:8000/docs.

In /docs, select `Try it out`, enter the required inputs, and select `Execute` to test the following:

1. Call `GET /health`.
2. Upload `fixtures/sample_data.csv`.
3. Use the returned ID to retrieve the dataset with `limit=2`.
4. Request an ID that does not exist.
5. Try uploading a file with an unsupported extension.

The upload should report six rows and two columns. Retrieval with `limit=2` should return two rows while `total_rows` remains six.

Expected Upload Response

```
{
  "id": 1,
  "filename": "sample_data.csv",
  "rows": 6,
  "columns": 2,
  "column_names": ["student_id", "score"]
}
```

The ID may differ after earlier uploads. Because score contains unknown, its values are returned as strings.

Example Server Logs

Your server logs should include information similar to the following.

```
10:15:02 DEBUG    src.storage — Stored dataset: id=1, rows=6
10:15:02 INFO     main — Uploaded dataset: id=1, filename=sample_data.csv,
rows=6
10:15:05 DEBUG    src.storage — Retrieved dataset copy: id=1, rows=6
10:15:05 INFO     main — Retrieved dataset: id=1, returned_rows=2
10:15:08 WARNING  main — Dataset not found: 999
10:15:10 WARNING  main — Unsupported file format: .txt
```

These are application logs. Uvicorn also displays its own server and access logs.

Expected Status Codes

Situation	Status
Successful health, upload, or retrieval	200
Unsupported or unreadable CSV upload	400
Dataset ID not found	404
Invalid parameter, such as limit=0	422

After Completing Part 1

Commit and push your work:

```
git status
git add .
git commit -m "Add API foundation and in-memory dataset storage"
git push -u origin main
```